AI/ML HACKATHON — PROBLEM STATEMENT

Smart Chargesheet Review & Summarisation Assistant

Team Size: Max 3 Members

---

1. Background

Police Chargesheets (Case Diaries / □□□□□ □□□□□□) in India are lengthy multi-page documents written in Hindi. They contain incident narratives, witness statements, seizure lists, and legal sections — often spanning 40+ pages.

Two problems make these documents hard to work with:

- Reviewing them manually is time-consuming for police officers and public prosecutors.
- Critical procedural documents (FSL reports, MLC certificates, sanction orders) are frequently missing — which can weaken a case during trial.

Mission

Build an AI assistant that ingests raw Hindi chargesheet text, generates a structured case summary, classifies the crime type, and flags missing procedural documents — all through a programmatic API pipeline.

2. Input Data

You will be given messy text from real Hindi Case Diaries. Expect:

- OCR errors and broken character sequences
- Broken two-column tables rendered as flat unstructured text
- Code-mixed Hindi/English content (e.g., "SANDISK 32 GB □□□□ □□□□ □□□")
- Inconsistent whitespace and non-standard date formats

Your pipeline must handle this noise. Pre-process intelligently — do not rely on the LLM to fix raw OCR garbage.

3. Ground Truth Checklists

The JSON below is authoritative. Load it as a static config file (`checklists.json`). Do not hardcode checklist items in your logic.

Crime Type 1 — Theft / Robbery

```
{
  "theft_robbery": {
    "display_name": "Theft / Robbery",
    "typical_sections": ["IPC 379", "IPC 380", "IPC 392"],
    "required_items": [
      "FIR number and date",
      "Police station name",
      "Place and time of occurrence",
      "Details of complainant and accused",
      "Description of stolen property and value",
      "Recovery/seizure memo of property",
      "Witness statements",
      "Site plan / spot inspection memo",
      "Arrest memo (if arrested)",
      "Chain of custody of recovered items"
```

```
    ]
  }
}
```

Crime Type 2 — Assault / Hurt

```
{
  "assault_hurt": {
    "display_name": "Assault / Hurt",
    "typical_sections": ["IPC 323", "IPC 324", "IPC 325"],
    "required_items": [
      "FIR details (number, date, PS)",
      "Victim and accused details",
      "Medical Legal Case (MLC) report",
      "Injury certificate with nature and number of injuries",
      "Photographs of injuries (if available)",
      "Description and seizure memo of weapon (if recovered)",
      "Doctor's opinion on nature of injuries",
      "Witness statements about assault"
    ]
  }
}
```

Crime Type 3 — Cyber Fraud

```
{
  "cyber_fraud": {
    "display_name": "Cyber Fraud / Online Cheating",
    "typical_sections": ["IPC 419", "IPC 420", "IT Act 66C", "IT Act 66D"],
    "required_items": [
      "Details of platform and mode of fraud",
      "Transaction records (bank/UPI statements)",
      "Screenshots or logs of chats/emails/calls",
      "Seizure memo of digital devices",
      "Requests to banks/companies for KYC and logs",
      "Responses from banks/companies",
      "Cyber-forensic / FSL report (if available)",
      "Chain of custody of digital evidence",
      "Linking of digital identifiers (IP/IMEI) to accused"
    ]
  }
}
```

Crime Type 4 — NDPS (Drugs / Narcotics)

```
{
  "ndps": {
    "display_name": "NDPS (Drugs / Narcotics)",
    "typical_sections": ["NDPS 20", "NDPS 21", "NDPS 22"],
    "required_items": [
      "Exact place, date and time of seizure",
      "Description of substance and quantity (small/intermediate/commercial)",
      "Search memo and seizure memo with seal details",
```

```
      "Compliance note for relevant NDPS provisions (e.g. Sections 42, 50)",
      "Weighment panchnama",
      "Sample drawing memo and sample details",
      "Chain of custody record (seizure to malkhana to FSL)",
      "Malkhana register entry",
      "FSL report",
      "Statements of independent witnesses"
    ]
  }
}
```

4. Stages

There are two stages. Stage 1 is required. Stage 2 is optional bonus work — attempt it only after Stage 1 is solid.

Stage 1 — Core Pipeline (Required)

Given raw Hindi chargesheet text, your system must produce three outputs:

Output A — Structured Case Summary

- FIR number, date, Police Station name
- Accused name(s) and Victim name(s)
- Comprehensive incident facts
- Legal sections applied (e.g., IPC 379, NDPS 20)

Output B — Crime Classification

- Classify into one of: **Theft/Robbery, Assault/Hurt, Cyber Fraud, NDPS**
- If no confident match, return a graceful fallback:
  `{ "crime_type": "UNKNOWN", "reason": "No matching sections found" }`

Output C — Missing Items Checklist

Compare extracted content against the JSON schema for the detected crime type. For each required item, output:

- ✅ `PRESENT` — evidence clearly found in the text and give its detail in brief
- ❌ `MISSING` — item not detected
- ⚠ `PARTIAL` — item referenced but appears incomplete/ where it's referenced and what's the missing part

Technical Requirements

- All LLM calls must be programmatic via API — no manual copy-pasting into a chat UI.
- Pre-process noisy text before sending to the LLM (normalize whitespace, strip OCR garbage, detect language blocks).
- Use structured output prompting — ask the LLM to return JSON, then validate it in your code.
- (optional) Try using the LLM as an extraction engine. Your own **deterministic code** do the checklist comparison against the JSON — not the LLM.

Stage 2 — Bonus: Smarter Extraction (Optional)

Attempt this only after Stage 1 is fully working. You can do 2A, 2B, or both — each is independent.

2A — Named Entity Recognition

Build an NER layer that extracts structured legal entities from the chargesheet text. Entity types to extract:

`PERSON`	Accused, victim, witness, officer names
`LEGAL_SECTION`	IPC 379, NDPS 20, IT Act 66C, etc.
`DATE_TIME`	Date of occurrence, FIR date, arrest date
`LOCATION`	Crime scene address, police station name
`DOCUMENT`	FIR, MLC report, seizure memo, FSL report
`AMOUNT`	Stolen property value, drug quantity

Minimum output format:

```
{ "entities": [
    { "text": "□□□ □□□□□", "type": "PERSON", "role": "ACCUSED" },
    { "text": "IPC 379",   "type": "LEGAL_SECTION" },
    { "text": "12/03/2024","type": "DATE_TIME", "event": "FIR_DATE" }
  ]
}
```

2B — Semantic Similarity Scoring

Stage 1 likely uses keyword matching to detect checklist items. But what if the chargesheet says "□□□□□□□ □□□□□ □□□□ □□□" instead of the exact phrase "Weighment panchnama"? A keyword check fails. A semantic check succeeds.

Replace or augment your Stage 1 checklist detection with a similarity-based approach:

- Represent each `required_item` as a vector using TF-IDF or sentence embeddings (e.g., Gemini embedding API or any open-source model).
- Represent each sentence/paragraph of the chargesheet as a vector in the same space.
- Compute cosine similarity for each required_item against all chargesheet sentences. Use a threshold (e.g., 0.65) to determine PRESENT / MISSING.
- Display the top-matching sentence alongside each PRESENT item.

Minimum output format:

```
{ "checklist": [
    {
      "item":             "Weighment panchnama",
      "status":           "PRESENT",
      "similarity_score": 0.81,
      "matched_text":     "□□□□□□□ □□□□□ □□□□ □□□ □□□□□□ 10/03/2024"
    }
  ]
}
```

5. Constraints

Team Size	Maximum 3 participants per team.
API	Google AI Studio Free Tier (Gemini 1.5 Flash / Pro recommended). Other free open-source APIs allowed with prior written consent.
API Keys	Each team manages their own keys. Do NOT hardcode keys — use environment variables.
LLM calls	All calls must be programmatic. No manual copy-pasting into ChatGPT, Gemini web UI, or any chat interface.
Language / Stack	Python or Node.js strongly recommended. Any language is fine as long as API calls are programmatic.
UI Time Budget	Max 1–2 hours on frontend. Use Streamlit, Gradio, or a simple React template. Do not over-engineer the UI.
Submission	GitHub repo with README, setup instructions, and short demo video or screenshots.

Judging Notes

- Going deep on Stage 1 + 2 correctly scores higher than rushing to Stage 2 with broken Stage 1 output.
- Judges will test with a fresh chargesheet at evaluation — your system must work on unseen input.
- Code quality matters — judges will review your pipeline architecture and checklist comparison logic.

6. Deliverables

1. Working application — accepts Hindi chargesheet text and displays all completed stage outputs.
2. **GitHub repository** — clean, commented code with a README containing setup steps and stage completion status.
3. `checklists.json` — included in the repo as a static config file, referenced by your code.
4. 3-minute live demo — judges will provide a fresh chargesheet during evaluation.
5. Stage completion table in your README — a simple list of which stages/features your team completed.

7. Contacts

For questions, API consent, or clarifications during the hackathon:

Keshav Trivedi	+91 84012 29069
Shivapreetham H S	+91 98459 20244
Hemanvitha Pullela	+91 89190 34799

Build something that would genuinely help Indian courts and law enforcement. Good luck.